



# BITS Pilani presentation

**BITS Pilani**  
Pilani Campus

Prof. Narasimha Bolloju  
CSIS Department, Hyderabad



# **SS ZG622**

## **Software Project Management**

### **Lecture No. 4**

# Recap



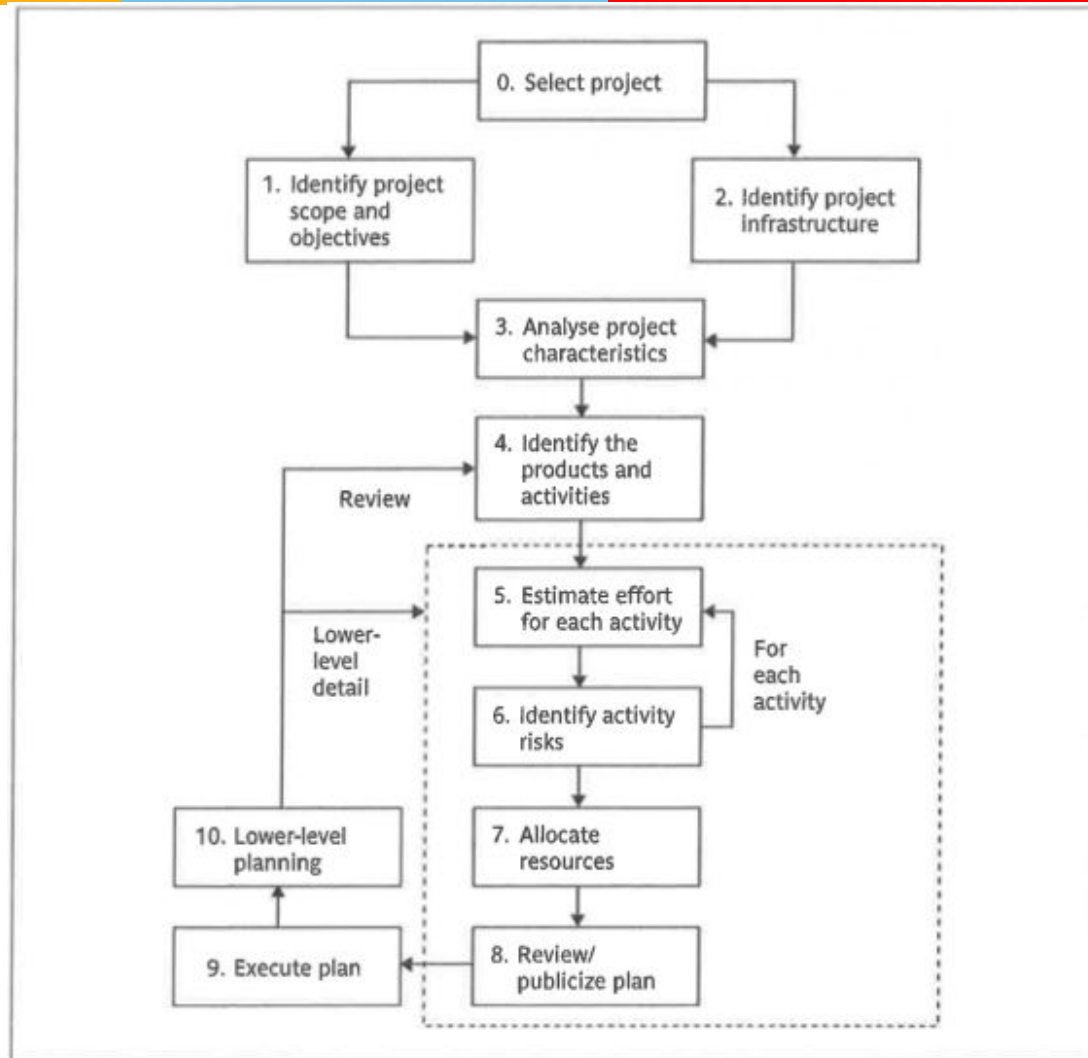
- Challenges in software project management (20% failure; 30% successful and 50% challenged)
- Need for understanding project and project management goals and objectives before planning and execution
- Plan-driven vs value-driven projects
- Project charter and its importance

# Recap



| Challenge                         | Impact                                           | Within PM's Control? |
|-----------------------------------|--------------------------------------------------|----------------------|
| Unclear or Changing Requirements  | Scope creep, delays, misaligned deliverables     | Partially            |
| Inaccurate Effort/Cost Estimation | Budget overruns, missed deadlines                | Yes                  |
| Poor Stakeholder Engagement       | Misaligned expectations, lack of buy-in          | Yes                  |
| Scope Creep                       | Increased costs, delayed timelines               | Partially            |
| Inadequate Risk Management        | Unforeseen issues, project failures              | Yes                  |
| Team Collaboration Issues         | Reduced productivity, conflicts                  | Yes                  |
| Resource Constraints              | Delays, compromised quality                      | Partially            |
| Schedule Delays                   | Missed deadlines, increased costs                | Partially            |
| Technical Debt Accumulation       | Long-term maintenance issues, rework             | Yes                  |
| Inadequate Quality Assurance      | Defective deliverables, customer dissatisfaction | Yes                  |

**innovate**      **achieve**      **lead**



# Product & Process Measures and Effort estimation

---



1. Product/process and direct/indirect measures
2. Size- and function-oriented metrics
3. Effort estimation approaches
4. Effort estimation using function points

# Product and Process Measures



|                 | Direct Measures                                                                                                                                                                                                                                                                                                                                                                                | Indirect Measures                                                                                                                                                                                                                                                                                             |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Product Metrics | <p>Lines of Code (LOC or KLOC): A measure of the size of the software by counting the number of lines in the source code.</p> <p>Cyclomatic Complexity: A measure of the complexity of a program by calculating the number of linearly independent paths through the source code.</p> <p>Defect Density: Number of defects per unit of code (e.g., bugs reported per 1,000 lines of code).</p> | <p>Customer Satisfaction: Often measured through surveys or Net Promoter Score (NPS), reflecting user satisfaction with the product.</p> <p>Maintainability Index: A measure that combines several factors (e.g., LOC, complexity) to predict the ease of future maintenance.</p>                             |
| Project Metrics | <p>Earned value (EV) with schedule variance and cost variance.</p> <p>Cycle Time: The time taken to complete a specific process or task from start to finish.</p>                                                                                                                                                                                                                              | <p>Developer Productivity: Often inferred from metrics like LOC per day or features completed per sprint, indirectly measuring efficiency.</p> <p>Risk Exposure: A qualitative measure of potential risks that could impact the project, often assessed through expert judgment or risk assessment tools.</p> |

# Direct vs Indirect measures



- Direct Measures are quantifiable and typically involve straightforward data collection. They focus on measurable aspects like code, time, or cost.
- Indirect Measures are qualitative/quantitative and involve interpretation or inference from other measures. They focus on outcomes or effects that are influenced by multiple factors.



# Activity W04#1

---



Connecting goals, objectives, metrics, and  
functional specifications  
*(enter your answers in [this document](#))*

# Size-oriented metrics



- Metrics are derived by normalizing quality and/or productivity measures by considering the size of the software produced in KLOC (thousand lines of code)
- Example metrics include
  - Errors per KLOC
  - Defects per KLOC
  - Cost per KLOC
  - Errors per person-month
  - KLOC per person-month
  - Cost per page of documentation
- KLOC measurements
  - Are dependent on the programming language
  - Penalize well-designed but short programs
  - Cannot easily accommodate nonprocedural languages
  - Require a level of detail that may be difficult to achieve



# Function-oriented metrics

---

- Function-oriented metrics (e.g., function points FP) use a measure of the functionality delivered by the application as a normalized value
  - Errors per FP
  - Defects per FP
  - Cost per FP
  - Pages of documentation per FP
  - FP per person-month

# Use of Metrics

---

- Product and project metrics are quantitative measures to gain insight into the efficiency of the software process and the projects
- Project metrics enable project manager to
  - Assess status of ongoing project
  - Track potential risks
  - Uncover problem are before they go critical
  - Adjust work flow or tasks
  - Evaluate the project team's ability to control quality of software work products

# Activity W04#2

---



Read [these functional specification](#) and  
give a rough estimate of effort [in person-months](#)  
for developing the **Mux-Core sales system**

# Effort estimation - approaches

---



- Expert judgment
- Analogous estimation
- Parametric estimation
- Bottom-up estimation
- Top-down estimation
- Wideband Delphi technique
- **Function points**
- Use-case points
- *COCOMO (Constructive Cost Model)*
- *Story points (Planning Poker)*



# Need for multiple approaches

---

- In case of large projects, we cannot depend on one approach to estimation, because there could be some aspects which might have been overlooked
- Better to use multiple estimation techniques and compare the results
- If the difference is large then it is likely that some aspects have been overlooked
- Understand the reason for the difference and re-estimate until results converge (say within 10%)

# Activity W04#3

---



Now, estimate the effort required for each function listed in [this Google Sheet](#) and then give your total effort estimate



# Function point (FP) technique



- Each requirement (e.g., selling a ticket at counter) is mapped to several functions and logical data entities/files
- The calculation of size is based on functions supported by the software and the data files used by the software
- Functions are of 3 types
  - External inputs
  - External outputs
  - External queries
- Data files are of 2 types
  - Internal logical files
  - External logical files

# Types of Functions

---

- **External Inputs (EI):** functions that modify the data of the system.
  - Examples: Create Purchase order, Withdraw money from ATM, Update price of a product
- **External Outputs (EO):** functions that send data to external systems or do some processing of data and provide to end users.
  - Examples: Exporting data to other systems via files or messages, Generating reports involving analysis such as average
- **External Queries (EQ):** inquiry functions that extract data from the system and present to user.
  - Examples: Show availability of seats in a flight, generating a list of all orders to be shipped tomorrow

# Types of Data

---

- Internal Logical Files: files (database entities/tables or flat files) maintained by the system and that reside within the system.
  - Example: Employee file, Orders file, Payment file, Inventory issue file
- External Interface Files: data files (database tables or flat files) maintained outside the system but needed by the system under development.
  - Example: Schedule of flights of other airlines, Interest rate offered by different banks

# Functions Points

---

## Steps to calculate Function points

1. Assign FPs to each Function & each File based on its complexity
2. Total the FPs. This is called Unadjusted FP or Raw FPs
3. Multiply the Unadjusted FPs by an adjustment factor. The adjustment factor is determined / based on 14 characteristics of the system

# FPs for Functions & Files

Different types of functions and files, and their complexity for a given requirement to be implemented

| Program Characteristic                                                        | Low Complexity | Medium Complexity | High Complexity |
|-------------------------------------------------------------------------------|----------------|-------------------|-----------------|
| External Inputs                                                               | ___ × 3        | ___ × 4           | ___ × 6         |
| External Outputs                                                              | ___ × 4        | ___ × 5           | ___ × 7         |
| External Queries                                                              | ___ × 3        | ___ × 4           | ___ × 6         |
| Internal Logical Files                                                        | ___ × 4        | ___ × 10          | ___ × 15        |
| External Interface Files                                                      | ___ × 5        | ___ × 7           | ___ × 10        |
| Unadjusted Function Point (UPFs) = Sum of EI FP, EO FP, EQ FP, ILF FP, EIF FP |                |                   |                 |

# Function point (FP) estimation technique

---



- FP technique is used in the early stages of the project to determine the size of the project based on software requirements and refined in later stages based on the design details
- The size of software is measured in Function Points
- Once the size is estimated in FPs, we can convert FPs to Effort, based on historical data if available (otherwise, established productivity data can be used)

# Function point estimation



| Function Count                     |                          |            |       |        |          |
|------------------------------------|--------------------------|------------|-------|--------|----------|
| Item                               | Item Description         | Complexity | Count | Weight | Weighted |
|                                    |                          |            |       |        | Count    |
| 1                                  | External Inputs          | Simple     | 0     | 3      | 0        |
|                                    |                          | Average    | 0     | 4      | 0        |
|                                    |                          | Complex    | 0     | 6      | 0        |
| 2                                  | External Outputs         | Simple     | 0     | 4      | 0        |
|                                    |                          | Average    | 0     | 5      | 0        |
|                                    |                          | Complex    | 0     | 7      | 0        |
| 3                                  | External Inquiries       | Simple     | 0     | 3      | 0        |
|                                    |                          | Average    | 0     | 4      | 0        |
|                                    |                          | Complex    | 0     | 6      | 0        |
| 4                                  | Internal Logical Files   | Simple     | 0     | 7      | 0        |
|                                    |                          | Average    | 0     | 10     | 0        |
|                                    |                          | Complex    | 0     | 15     | 0        |
| 5                                  | External Interface Files | Simple     | 0     | 5      | 0        |
|                                    |                          | Average    | 0     | 7      | 0        |
|                                    |                          | Complex    | 0     | 10     | 0        |
| Total Weighted Function Count (FC) |                          |            |       |        | 0        |

# Function point estimation

| Complexity Factor                                          |                                            |              |
|------------------------------------------------------------|--------------------------------------------|--------------|
| Factor                                                     | Description                                | Rating       |
|                                                            |                                            | 0=Irrelevant |
|                                                            |                                            | 5=Essential  |
| F1                                                         | Reliability and backup recovery            |              |
| F2                                                         | Data communications                        |              |
| F3                                                         | Distributed processing                     |              |
| F4                                                         | Performance                                |              |
| F5                                                         | Operate on existing system                 |              |
| F6                                                         | On-line data entry                         |              |
| F7                                                         | Data entry over multiple screens           |              |
| F8                                                         | Master files updated on-line               |              |
| F9                                                         | Complex inputs, outputs, files & inquiries |              |
| F10                                                        | Complex internal processing                |              |
| F11                                                        | Code needs to be reusable                  |              |
| F12                                                        | Need conversion and installation           |              |
| F13                                                        | Multiple installations of the system       |              |
| F14                                                        | Easy to change and use                     |              |
| Complexity Factor (CF) = sum of ratings                    |                                            | 0            |
| Function Points                                            |                                            |              |
| Function Points (FP) = $FC \times (0.65 + 0.01 \times CF)$ |                                            | 0            |



**Estimate the effort using function point method for these specifications of Mux-Core system**

Instructions:

- Make a copy of this document “[estimating function points](#)”
- Estimate function points for F2 to F6 (for F1 it is already estimated)

# FP productivity based on programming language (only an example)



| Language     | Hours per FP | FP per month |
|--------------|--------------|--------------|
| ASP*         | 06.1         | 28           |
| Visual Basic | 08.5         | 20           |
| Java         | 10.6         | 16           |
| SQL          | 10.8         | 16           |
| C++          | 12.4         | 14           |
| C            | 13.0         | 13           |
| PL/1         | 14.2         | 12           |
| C#           | 15.5         | 11           |
| COBOL        | 16.8         | 10           |
| ABAP         | 19.9         | 9            |

[http://www.webratio.com/website/documentation/Case\\_Study\\_Productivity\\_with\\_WebRatio.pdf](http://www.webratio.com/website/documentation/Case_Study_Productivity_with_WebRatio.pdf)

# Additional slides on effort estimation

# Example of historical data



**Table 7-1** Examples of Quantities That Can Be Counted for Estimation Purposes

| Quantity to Count        | Historical Data Needed to Convert the Count to an Estimate                                                                                                                                                                                                         |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Features                 | <ul style="list-style-type: none"><li>■ Average effort hours per feature for development and/or testing</li></ul>                                                                                                                                                  |
| Use cases                | <ul style="list-style-type: none"><li>■ Average total effort hours per use case</li><li>■ Average number of use cases that can be delivered in a particular amount of calendar time</li></ul>                                                                      |
| Stories                  | <ul style="list-style-type: none"><li>■ Average total effort hours per story</li><li>■ Average number of stories that can be delivered in a particular amount of calendar time</li></ul>                                                                           |
| Engineering requirements | <ul style="list-style-type: none"><li>■ Average number of engineering requirements that can be formally inspected per hour</li><li>■ Average effort hours per requirement for development/test/documentation</li></ul>                                             |
| Function Points          | <ul style="list-style-type: none"><li>■ Average development/test/documentation effort per Function Point</li><li>■ Average lines of code in the target language per Function Point</li></ul>                                                                       |
| Change requests          | <ul style="list-style-type: none"><li>■ Average development/test/documentation effort per change request (depending on variability of the change requests, the data might be decomposed into average effort per small, medium, and large change request)</li></ul> |
| Web pages                | <ul style="list-style-type: none"><li>■ Average effort per Web page for user interface work</li><li>■ Average whole-project effort per Web page (less reliable, but can be an interesting data point)</li></ul>                                                    |

# Data collection

- Collect effort for all activities and all items (screen, report, etc.) including
  - Requirement gathering
  - Architecture
  - Detailed design
  - Coding & Unit testing
  - Design of test cases
  - Integration testing
  - System testing
  - User acceptance testing
  - Project Management
  - User documentation
  - Reviews of requirement, design, code, test cases, user manual, etc.
- Collect data while project is underway. It is difficult to go back six months in the project to try to collect data
- If you do not have historical data, use data from current project, as a basis to estimate the remainder of the project

# Calibration



- Based on data collected, we can calibrate our productivity, ie. Effort for unit size
- Calibration examples
  - Our team takes 8 hours for design & 24 hours to code one screen
  - Our testers take 6 hours to create a test case
  - Our maintenance team takes 4 hours to fix a defect

# Factors influencing effort estimation



## 1. Project size:

- Project size has the biggest influence on effort estimation
- Effort increases exponentially (not linearly) as project size increases because there is much more coordination in larger projects (this is opposite of 'Economies of scale' concept)

## 2. Kind of software being developed

- Avionics, embedded systems and System software are much more complex than business systems and internet systems

## 3. Personal factors impact estimation

- Capability of requirements analyst, capability of programmer, change of team members, level of domain knowledge, platform experience, team cohesion

# Why Opt for FP?

---

- Used readily countable characteristics that are determined early in the software process
- Does not “penalize” inventive (short) implementations that use fewer LOC than other more clumsy versions
- Makes it easier to measure the impact of reusable components





# Function Point Controversy

---

- Proponents claim that
  - FP is programming language independent
  - FP is based on data that are more likely to be known in the early stages of a project, making it more attractive as an estimation approach
- Opponents claim that
  - FP requires some “sleight of hand” because the computation is based on subjective data
  - FP has no direct physical meaning...it’s just a number

# Determining complexity of EI, EO & EQ



- Complexity of functions is determined based on number of Data Element Types referenced and File Types referenced, by the function
  - Example #1: The function “Create purchase order” (EI), updates the fields – PO#, Date, Description, PO amount, Customer ID, Customer name, etc. These are the DETs maintained by the function
  - Example #2: The report “Generate Sales analysis report” (EO) displays the following fields on the report – Product ID, Product name, City, Month, Total quantity sold, Total Sales revenue, Name of Salesman. These fields are the DETs of this function. In order to perform this function, it uses 3 files namely – Sales file, Product file and Salesman file. These are the File Types referenced

# Determining complexity of EI, EO & EQ



- Here is the matrix to determine the complexity

**Data Element Types (DETs)** – user-recognizable data elements that are maintained as ILFs by the EIs, appear in the EOs, EQs (data fields, buttons, search).

**File Types Referenced (FTRs)** – all ILFs and EIFs referenced or maintained during the processing of the EIs/EOs/EQs (tables, flat files).

| EI      | 1-4 DETs | 5-15 DETs | 16 + DETs |
|---------|----------|-----------|-----------|
| 0-1 FTR | Low      | Low       | Average   |
| 2 FTRs  | Low      | Average   | Average   |
| 3+ FTRs | Average  | High      | High      |

| EO/EQ    | 1-5 DETs | 6-19 DETs | 19+ DETs |
|----------|----------|-----------|----------|
| 0-1 FTR  | Low      | Low       | Average  |
| 2-3 FTRs | Low      | Average   | High     |
| 4+ FTRs  | Average  | High      | High     |

# Determining complexity of files



- Complexity of files is determined based on 2 characteristics of the file namely - Data Element Types (DET) and Record Element Types (RET)
- The meaning of DET & RET is explained below:
- Example:
  - Employee file can have the following logical groupings
    - Basic data of employee such as Name, Date of birth, Address, etc.
    - Past employment info such as Name of company, Start date, end date, Last designation, etc.
    - Past project info such as Name of project, From date, To date, Role in project, etc.
  - In the above example,
    - the DETs are Name, date of birth, Address, Name of company, From date, etc.
    - We have 3 types of logical grouping of data namely - Basic data group, Past employment info group, Past project info group. Hence we have 3 RETs

# Determining complexity of LIF & EIF



- Here is the matrix to determine the complexity

**Data Element Types (DETs)** – Unique, user-recognizable, non-repeating fields or attributes, including foreign key attributes that enter the boundary of the subsystem or application (fields).

**Record Element Types (RETs)** – Logical sub-groupings based on the user's view of the data (0 RET = 1 RET).

| ILF/EIF  | 1-19 DETs | 20-50 DETs | 51 + DETs |
|----------|-----------|------------|-----------|
| 1 RET    | Low       | Low        | Average   |
| 2-5 RETs | Low       | Average    | High      |
| 6+ RETs  | Average   | High       | High      |

# Adjustment factor

- Adjustment factor depends on the rating of 14 system characteristics
- We need to rate each characteristic on a scale of 0-5
- The adjustment factor is calculated using a formula shown below

|                                                                                       |                               |       |
|---------------------------------------------------------------------------------------|-------------------------------|-------|
| F1                                                                                    | Reliable back-up and recovery | 0 – 5 |
| F2                                                                                    | Data communications           | 0 – 5 |
| F3                                                                                    | Distributed functions         | 0 – 5 |
| F4                                                                                    | Performance                   | 0 – 5 |
| F5                                                                                    | Heavily used configuration    | 0 – 5 |
| F6                                                                                    | Online data entry             | 0 – 5 |
| F7                                                                                    | Operational ease              | 0 – 5 |
| F8                                                                                    | Online update                 | 0 – 5 |
| F9                                                                                    | Complex interface             | 0 – 5 |
| F10                                                                                   | Complex processing            | 0 – 5 |
| F11                                                                                   | Reusability                   | 0 – 5 |
| F12                                                                                   | Installation ease             | 0 – 5 |
| F13                                                                                   | Multiple sites                | 0 – 5 |
| F14                                                                                   | Facilitate change             | 0 – 5 |
| <b>Value adjustment factor (VAF) = <math>0.65 + 0.01 * \text{Sum}(F1, F14)</math></b> |                               |       |
| <b>Adjusted function point (AFP) = <math>\text{UFPs} * \text{VAF}</math></b>          |                               |       |

# Object based estimation

---

- This is a flexible estimation approach that can be adapted to different types of projects
- For example, some projects have screens & reports as objects. Others have configurations settings as objects (ex, ERP package implementation), others have change requests as objects
- This technique determines the size of project based on number of objects and their complexity
- Please note that this is not the same as Object Point technique



# Object based estimation

---

## Steps:

1. List all objects of different categories
2. Assign points to each object based on its complexity (Low, Medium, High)
3. Determine the total points
4. Finally convert the Points to effort based on Calibrated Historical data

**Note:** The classification of objects and the way to determine their complexity needs to be done by each organization depending on the nature of projects executed



# Assigning points, based on complexity of objects: Example

## Point based on complexity

| Object type | Complexity -> | Low | Medium | High |
|-------------|---------------|-----|--------|------|
| Screen      |               | 2   | 4      | 8    |
| Report      |               | 1   | 2      | 3    |
| Interface   |               | 2   | 4      | 8    |

Points will differ from organization to organization. It has to be based on historical data of effort needed to develop & test these objects



# Object based estimation: Example

| Screens             | Name of screen             | Complexity | Points |
|---------------------|----------------------------|------------|--------|
|                     | Account maintenance        | High       | 8      |
|                     | Funds transfer             | Medium     | 4      |
|                     | Pay bills                  | Low        | 2      |
| Reports             | Name of report             | Complexity | Points |
|                     | Generate monthly statement | High       | 3      |
|                     | Print last 10 transactions | Medium     | 2      |
| Interfaces          | Name of interface          | Complexity | Points |
|                     | Send statement via email   | Low        | 2      |
|                     | Receive government rules   | Medium     | 4      |
| Total Object points |                            |            | 25     |

# Parameters that determine complexity of objects: Example



- Given below are some parameters that can be used to determine complexity of objects:
  - Screen complexity: can be determined based on number of fields on the screen, number of command buttons,
  - Report complexity: can be determined based on number of fields in the report, number of data files needed
  - Interface complexity: can be determined by the number of data items sent or received, number of data files needed

# Determining complexity of objects: Example



## Screen complexity determination: Example

|                           |       | Number of fields |         |         |
|---------------------------|-------|------------------|---------|---------|
|                           |       | <5               | 6 - 10  | > 10    |
| Number of command buttons | 1     | Simple           | Simple  | Medium  |
|                           | 2 - 3 | Simple           | Medium  | Medium  |
|                           | > 3   | Simple           | Complex | Complex |

# Summary



- Object based estimation is a flexible approach that can be adapted to different types of projects
- Projects need to identify suitable types of objects based on the type of project
- The size of projects are estimated based on number of objects and complexity of objects



# Estimating effort for different phases

- Once the size of the project is estimated using FPs or Object points, we can convert the size into effort, based on calibrated historical data
- We can then distribute the total effort across different phases of the project, again based on historical data
- For example:
  - 7% for Requirement gathering
  - 8% for Requirement analysis
  - 20% for architecture & design
  - 45% for construction
  - 15% for System testing
  - 5% for Project management

# Estimating effort: Other considerations

---

There may be other factors to consider:

- Configuration management: 2-5% of technical effort
- Requirement change: 1-4% effort per elapse month
- First time development with new language & tool: 20- 40% effort
- Contingency depending on risk severity & probability: 5-10%

**Note: All percentages are illustrative only. Actual percentages should be based on historical data**



# Estimation negotiation techniques

---

Project managers encounter situations where stakeholders ask for changes to estimation. Here are some ways to handle them:

1. If Marketing team tells you to reduce the cost or effort
  - Use historical data to justify the effort calculations
2. If more features are requested without adding more effort
  - Move some functions to version 2, or
  - Cut features altogether, if the business value is low
3. When you are asked to complete the project faster
  - Ask for more developers & testers & team leads, or
  - Add higher skilled technical staff, or
  - Ask what features are a 'must have' & what are not required